

EveCore

Instruction Set

Architecture

Harry H. Porter III
2 July 2024

Website: Blitz64.org/EveCore/

This Document: Blitz64.org/EveCore/InstructionSet.pdf

Author: HHPorter3@gmail.com

EveCore is an 8-bit computer architecture featuring:

- 8 registers of 8 bits each
- 16 bit program space (up to 64 KB of flash)
- 16 bit data space (up to 64 KB of RAM)
- 16 bit Stack Pointer register
- Instruction length of 1, 2, or 3 bytes
- 156 instructions
- Up to 256 I/O registers

This document describes the architecture and each of the instructions.

Table of Contents

1: Architectural Overview	4
1.1: Registers	4
1.2: Memory	5
1.3: Instructions	7
1.4: Example Program	8
1.5: Instruction Categories	9
1.6: Instruction Opcode Encodings	10
2: Numerical Opcode List	12
3: The Instructions	18
3.1: MOV - Register to Register Move	18
3.2: LD - Load From RAM	19
3.3: ST - Store to RAM	20
3.4: PUSH - Push Register Onto Stack	21
3.5: POP - Pop Stack Top into Register	22
3.6: MOVI - Move Immediate Into Register	23
3.7: ADD16 - Add 16 Bit Values	24
3.8: AND - Logical And	27
3.9: OR - Logical OR	29
3.10: XOR - Logical Exclusive Or	30
3.11: NOT - Logical Negation	31
3.12: ADD - Addition	32
3.13: ADDC - Addition with Carry	34
3.14: SUB - Subtract	36
3.15: SUBC - Subtract with Carry (Borrow)	38
3.16: NEG - Arithmetic Negation	39
3.17: CLR - Clear Register	41
3.18: SHL - Shift Left	42
3.19: SHLC - Shift Left With Carry	43
3.20: SHR - Shift Right	45
3.21: SHRC - Shift Right With Carry	46
3.22: INCR - Increment	47
3.23: DECR - Decrement	48
3.24: LDSP - Load Stack Pointer	49
3.25: LDCODE - Load Byte From Flash Memory	51
3.26: GOTOXY - Jump Indirect Through Register	53
3.27: CALL - Invoke Function	54
3.28: RET - Return From Function	57

3.29: IN - Read From I/O Register	58
3.30: OUT - Write To I/O Register	59
3.31: LDM - Read From RAM Using M	60
3.32: STM - Store Into RAM Using M	62
3.33: CMP - Compare A to B	63
3.34: BRANCH - Conditional GOTO	64
3.35: NOP Instruction	68
4: Assembly Code	69
4.1: Introduction	69
4.2: Source and Output Files	70
4.3: Assembler Directives	75
4.4: The .flash and .ram Directives	76
4.5: The .byte Directive	77
4.6: The .halfword and .word Directives	78
4.7: The .skip Directive	79
4.8: The .string Directive	80
4.9: The .equ Directive	81
5: Discussion	83
5.1: Motivation	83
5.2: Comparison With 6502 / 8051 / AVR	83
5.3: Functionality You Might Want to Add	84
5.4: What Was Left Out Of EveCore	85
Document Revision History	86
Change History Log	86

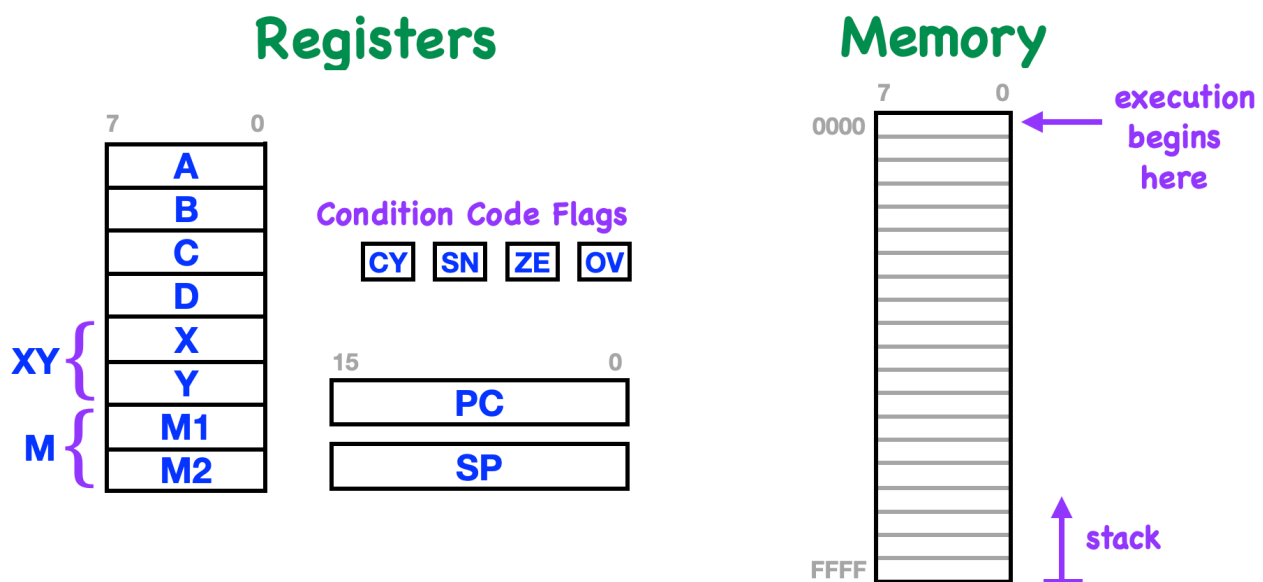
1: Architectural Overview

1.1: Registers

There are eight registers of 8 bits each, named **A**, **B**, **C**, **D**, **X**, **Y**, **M1**, and **M2**.

The **X** and **Y** registers are combined in some instructions and treated as a 16 bit register named **XY**. Likewise, the **M1** and **M2** registers are combined as the register named **M**.¹

The Stack Pointer register (**SP**) and Program Counter (**PC**) register are both 16 bits and can address up to 64 KB.²³



¹ The X and M1 registers form the most significant byte while the Y and M2 registers are used for the least significant byte.

² SP is always used to address RAM memory and 16 bits are needed if all 64 KB of RAM are implemented. However, if a specific implementation has less RAM, some upper bits of SP may be unneeded, inaccessible, and not implemented. Likewise, PC is always used to address program (flash) memory. If flash is less than 64 KB, any unneeded upper bits are not accessible and unneeded.

³ In this document, "KB" means 1,024 bytes.

There are four condition code bits:

- CY** — Carry Bit
- SN** — Sign Bit
- ZE** — Zero Bit
- OV** — Overflow Bit

The condition code bits are set by the ALU instructions and the CMP instruction and can be tested by the BRANCH instructions. The **CY** bit is used by the ADDC, SUBC, and SHIFT instructions.

The architecture is “accumulator-based”. All arithmetic and logical operations work on the **A** (“accumulator”) register, placing the result in **A**. Operands always come from the **A** and **B** registers.

Additional information and commentary is shown like this. While this may be of interest to some readers, it can be safely ignored.

The EveCore architecture is intended as an introductory microcontroller for hobbyists and engineering students. It may also be of use to designers needing a simple, open-source micro-core IP that can be easily and quickly tailored to a specific application.

1.2: Memory

The EveCore is designed to be implementable either as a “Harvard Architecture” or a “Von Neumann Architecture”.

With Harvard Architecture, the program and the data are kept in separate memories, while the Von Neumann Architecture has a single, unified memory in which data and instructions are both stored. Since operating systems load programs into memory as data and then execute those programs, an OS requires Von Neumann Architecture. Harvard Architecture computers are unable to host operating systems.

The Harvard Architecture is typical of many microcontroller designs. The program is kept in flash memory and the data in RAM. Before execution begins, the program must be loaded into flash memory. During execution, data may be stored in the RAM.

Program Memory	— FLASH memory
Data Memory	— Random Access Memory (RAM)

When implemented using the Harvard approach, the EveCore architecture accommodates up to 64 KB of program (flash) memory and up to 64 KB of data (RAM) memory. A particular implementation may have substantially less. For example, a specific implementation may have 8 KB of flash and 1 KB of RAM.

When implemented using a Von Neumann approach, the EveCore will have a single, unified memory of up to 64 KB, containing both programs and data.

The sizes of flash and RAM memories are always powers of 2. Addresses will “wrap around”, which is another way of saying that any unneeded upper bits of an address are simply ignored. For example, if the program memory is 4 KB, then an address of 12 bits is adequate, since $2^{12} = 4,096 = 4K$. The memory is addressed with values ranging over 0x0000...0x0fff. With wraparound, addresses 0x0007, 0x1007, 0x2007, 0x3007, ... all refer to the same location, since the upper 4 bits are ignored.

In this document, the following terminology is used:

	<u>bits</u>	<u>hex range</u>	<u>signed range</u>
byte	8	00 ... ff	-128 ... +127
halfword	16	0000 ... ffff	-32,768 ... +32,767
word	32	0000_0000 ... ffff_ffff	-2,147,483,648 ... +2,147,483,647

A **byte** is 8 bits, and can be represented in hex with a range of 0x00 ... 0xff

An 8 bit value can be used to encode such things as an ASCII character, a **signed** value, or an **unsigned** value.

An 8 bit signed value ranges over -128 ... +127

An 8 bit unsigned value ranges over 0 ... 255

Any 8 bit value can be interpreted as either a signed or an unsigned value or something else, as the programmer chooses.

The ADD instruction works identically on signed and unsigned values. In other words, there aren't different instructions for signed and unsigned. For example, 0x07 + 0xc8 is 0xcf. Interpreted as signed, this is $7 + -56 = -49$. Interpreted as unsigned, this is $7 + 200 = 207$.

Likewise, a 16 bit value can be represented with 4 hex digits with a range of 0x0000...0xffff

A 16 bit signed value ranges over -32,768 ... +32,767

A 16 bit unsigned value ranges over 0 ... 65,535

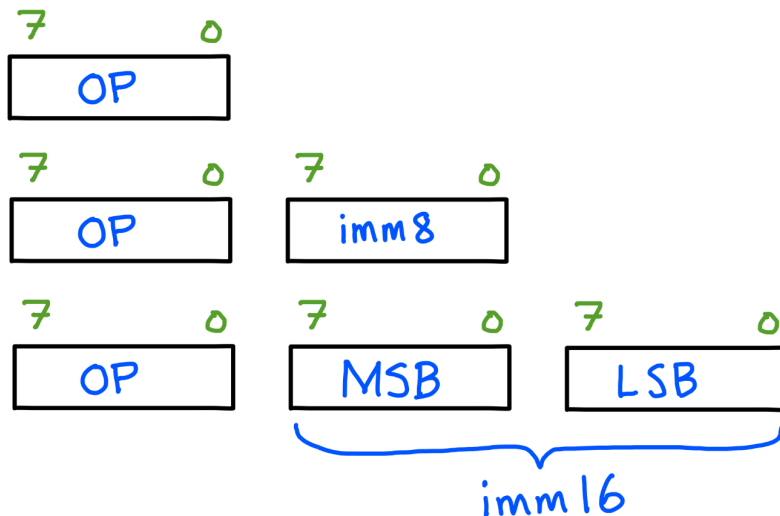
Caution: In the microcontroller world, the term “word” sometimes means 16 bits. Computer Science needs a standardized definition of “word” (like “byte” has), and 32 bits is preferable. This leaves the term “short” or “halfword” for 16 bits and “long” or “doubleword” for 64 bits.

1.3: Instructions

Instructions are either 1, 2, or 3 bytes in length. The first byte is always the **opcode**. Any registers involved are either implicit in the opcode or encoded in a field within the opcode byte.

For 2 byte instructions, the second byte is an 8 bit immediate value, referred to as **imm8**.

For 3 byte instructions, the second and third bytes comprise a 16 bit immediate value, referred to as **imm16**. The second byte is the most significant byte of **imm16** and the third byte is the least significant byte.



1.4: Example Program

Here is an example assembly language program for the EveCore architecture:

```
// Example program
.flash

Main:
    SP = 0x0000          // Stack begins at highest RAM addr
    XY = msg             // Print greeting
    call Print           //

Loop:
    A = in UART_recv     // Echo 1 byte on UART
    out UART_xmit = A    //
    B = 'q'              // if "q" then goto Main
    cmp A:B              //
    if == goto Main      //
    goto Loop            // else continue echoing

msg:    .string "\nEcho Program\nPlease type...\0"

// Print Function
//    passed: XY = pointer to NUL-terminated string in FLASH
//    uses:    A, B, XY
Print:
    A = code[XY]
    XY = XY + 1
    B = 0
    cmp A:B
    if == goto PrintExit
    out UART_xmit = A
    goto Print
PrintExit:
    ret

UART_xmit: .equ 0        // I/O register for sending
UART_recv: .equ 0        // I/O register for receiving
```

1.5: Instruction Categories

The instructions can be divided into the following general categories.

	<u>Opcode Byte</u>	<u>Byte2</u>	<u>Byte3</u>	<u>Assembly Syntax</u>
64	MOV R R			R_D = R_S
8	MOVI R	imm8		R = imm8
16	ALU op			A = A op B / A = op A
	CMP			cmp A:B
8	LD R	msb	lsb	R = [address]
8	ST R	msb	lsb	[address] = R
	LDM	msb	lsb	A = [M + imm16]
	STM	msb	lsb	[M + imm16] = A
	LDCODE			A = code[XY]
8	ADD16 R R	msb	lsb	[M/XY] = [M/XY/A/0] + imm16
16	BRANCH cond	msb	lsb	if cond goto address
	GOTOXY			goto XY
	LDSP	msb	lsb	SP = address
8	PUSH R			push R
8	POP R			pop R
	CALL	msb	lsb	call address
	RET			ret
	IN	imm8		A = in IOReg
	OUT	imm8		out IOReg = A
	NOP			nop

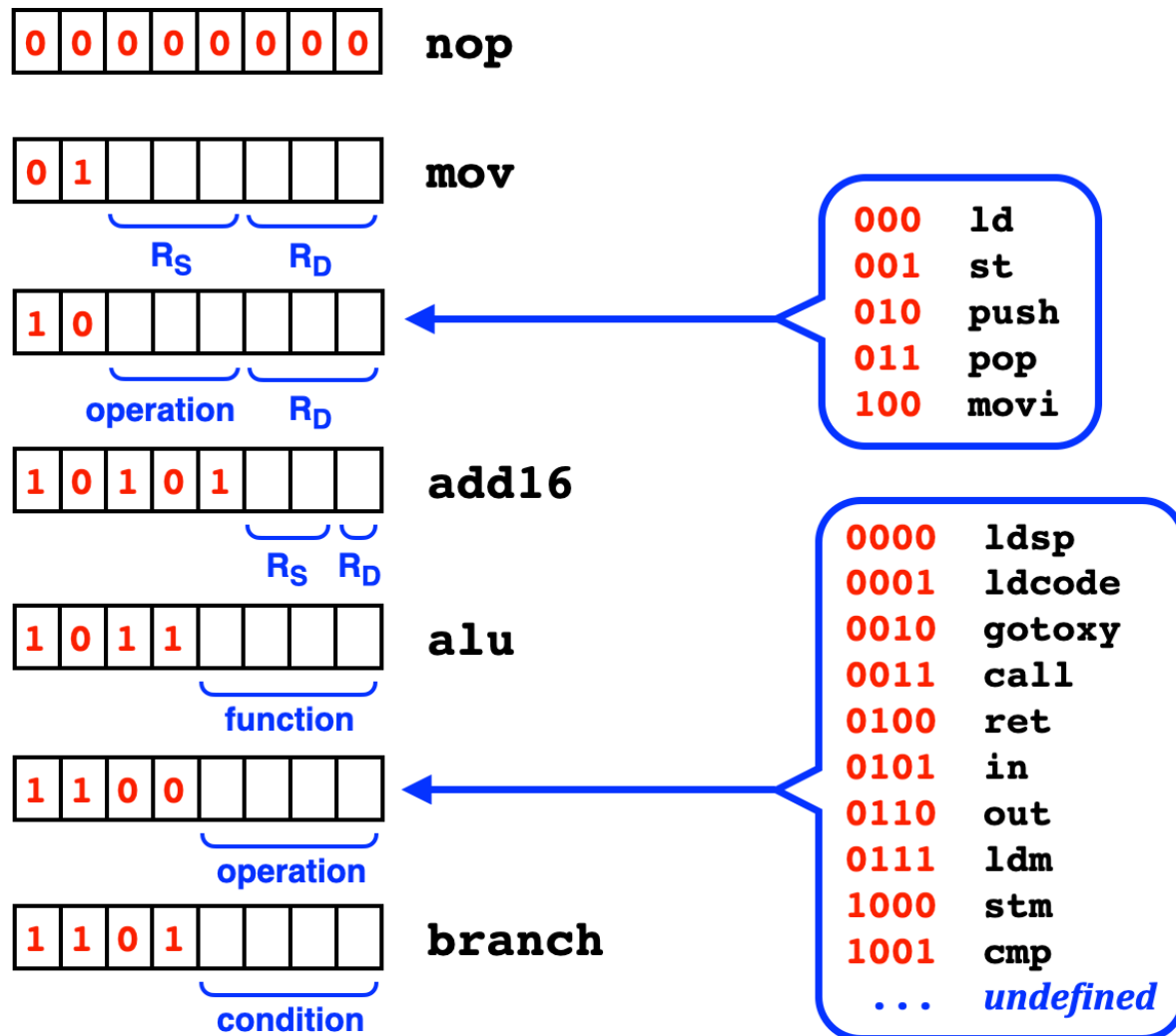
The number in the leftmost column indicates how many instructions are in the category. The column on the right both sketches the instruction operation as well as the syntax used by the assembler.

The material in **blue** shows syntactic options.

If the opcode byte is followed by a second byte, this is indicated with **imm8** or **IOReg**. If the opcode byte is followed by two bytes (msb-lsb), this is indicated with **imm16** or **address**.

Note that all “load” and “store” instructions — which contain addresses and access memory directly — use brackets in the assembly code. This syntax emphasizes that they touch memory.

1.6: Instruction Opcode Encodings



The 3 bit source and destination registers fields (R_S and R_D) are encoded in the obvious way:

000	A
001	B
010	C
011	D
100	X
101	Y
110	M1
111	M2

The **function** field in the **alu** instructions is encoded as follows:

0000	A = A & B	Bitwise AND
0001	A = A B	Bitwise OR
0010	A = A ^ B	Bitwise XOR
0011	A = !A	Bitwise NOT
0100	A = A + B	Add
0101	A = A + B + CY	Add with carry
0110	A = A - B	Subtract
0111	A = A - B - CY	Subtract with carry (i.e. borrow)
1000	A = -A	Signed negation
1001	A = 0	Clear
1010	A = A << 1	Shift left
1011	A = A <<< 1	Shift left through carry
1100	A = A >> 1	Shift right
1101	A = A >>> 1	Shift right through carry
1110	A = A + 1	Increment
1111	A = A - 1	Decrement

The encoding of the **condition** field in the **branch** instruction is given elsewhere in this document, where we describe that instruction. Likewise, the fields in the **add16** instruction are described where we describe that instruction.

All opcodes not otherwise mentioned are undefined. Specific implementations may choose to implement them as they see fit. Otherwise, the instruction is to be considered to be illegal and should not be used.

Some architectures have mechanisms to handle exceptions and/or traps of various kinds. In such systems, an illegal instruction typically invokes exception handling. To keep the EveCore simple, the behavior of illegal instructions is undefined.

One reasonable implementation is to freeze the core by throwing the state machine into a never-ending “error” state. In such an implementation, the programmer can insert the following in the code to effect a machine “halt”:

```
.byte 0xff
```

2: Numerical Opcode List

	Opcode			Size	Cond. Codes	Assembler Syntax	Notes
					C S Z V ⁴		
NOP	0000	0000	00	1		nop	
ILLEGAL	00--	----	01...3f	1			6
MOV	0100	0000	40	1		A = A	1
MOV	0100	0001	41	1		A = B	
MOV	0100	0010	42	1		A = C	
MOV	0100	0011	43	1		A = D	
MOV	0100	0100	44	1		A = X	
MOV	0100	0101	45	1		A = Y	
MOV	0100	0110	46	1		A = M1	
MOV	0100	0111	47	1		A = M2	
MOV	0100	1000	48	1		B = A	
MOV	0100	1001	49	1		B = B	1
MOV	0100	1010	4a	1		B = C	
MOV	0100	1011	4b	1		B = D	
MOV	0100	1100	4c	1		B = X	
MOV	0100	1101	4d	1		B = Y	
MOV	0100	1110	4e	1		B = M1	
MOV	0100	1111	4f	1		B = M2	
MOV	0101	0000	50	1		C = A	
MOV	0101	0001	51	1		C = B	
MOV	0101	0010	52	1		C = C	1
MOV	0101	0011	53	1		C = D	
MOV	0101	0100	54	1		C = X	
MOV	0101	0101	55	1		C = Y	
MOV	0101	0110	56	1		C = M1	
MOV	0101	0111	57	1		C = M2	
MOV	0101	1000	58	1		D = A	
MOV	0101	1001	59	1		D = B	
MOV	0101	1010	5a	1		D = C	
MOV	0101	1011	5b	1		D = D	1
MOV	0101	1100	5c	1		D = X	
MOV	0101	1101	5d	1		D = Y	
MOV	0101	1110	5e	1		D = M1	
MOV	0101	1111	5f	1		D = M2	

⁴ Blanks in the “Cond. Codes” column indicates the condition codes are unchanged by the instruction. A “0” indicates the condition code is cleared. A “1” indicates the condition code is set. A “C”, “S”, “Z”, or “V” indicates that the CY, SN, ZE, or OV condition code flag is set according to the result of the operation.

	<u>Opcode</u>		<u>Size</u>	<u>Cond. Codes</u> <u>C S Z V</u>	<u>Assembler</u> <u>Syntax</u>	<u>Notes</u>
MOV	0110	0000	60	1	X = A	
MOV	0110	0001	61	1	X = B	
MOV	0110	0010	62	1	X = C	
MOV	0110	0011	63	1	X = D	
MOV	0110	0100	64	1	X = X	1
MOV	0110	0101	65	1	X = Y	
MOV	0110	0110	66	1	X = M1	
MOV	0110	0111	67	1	X = M2	
MOV	0110	1000	68	1	Y = A	
MOV	0110	1001	69	1	Y = B	
MOV	0110	1010	6a	1	Y = C	
MOV	0110	1011	6b	1	Y = D	
MOV	0110	1100	6c	1	Y = X	
MOV	0110	1101	6d	1	Y = Y	1
MOV	0110	1110	6e	1	Y = M1	
MOV	0110	1111	6f	1	Y = M2	
MOV	0111	0000	70	1	M1 = A	
MOV	0111	0001	71	1	M1 = B	
MOV	0111	0010	72	1	M1 = C	
MOV	0111	0011	73	1	M1 = D	
MOV	0111	0100	74	1	M1 = X	
MOV	0111	0101	75	1	M1 = Y	
MOV	0111	0110	76	1	M1 = M1	1
MOV	0111	0111	77	1	M1 = M2	
MOV	0111	1000	78	1	M2 = A	
MOV	0111	1001	79	1	M2 = B	
MOV	0111	1010	7a	1	M2 = C	
MOV	0111	1011	7b	1	M2 = D	
MOV	0111	1100	7c	1	M2 = X	
MOV	0111	1101	7d	1	M2 = Y	
MOV	0111	1110	7e	1	M2 = M1	
MOV	0111	1111	7f	1	M2 = M2	1

	Opcode			Size	Cond. Codes	Assembler	Notes
					C S Z V	Syntax	
LD	1000	0000	80	3		A = [expr16]	
LD	1000	0001	81	3		B = [expr16]	
LD	1000	0010	82	3		C = [expr16]	
LD	1000	0011	83	3		D = [expr16]	
LD	1000	0100	84	3		X = [expr16]	
LD	1000	0101	85	3		Y = [expr16]	
LD	1000	0110	86	3		M1 = [expr16]	
LD	1000	0111	87	3		M2 = [expr16]	
ST	1000	1000	88	3		[expr16] = A	
ST	1000	1001	89	3		[expr16] = B	
ST	1000	1010	8a	3		[expr16] = C	
ST	1000	1011	8b	3		[expr16] = D	
ST	1000	1100	8c	3		[expr16] = X	
ST	1000	1101	8d	3		[expr16] = Y	
ST	1000	1110	8e	3		[expr16] = M1	
ST	1000	1111	8f	3		[expr16] = M2	
PUSH	1001	0000	90	1		push A	
PUSH	1001	0001	91	1		push B	
PUSH	1001	0010	92	1		push C	
PUSH	1001	0011	93	1		push D	
PUSH	1001	0100	94	1		push X	
PUSH	1001	0101	95	1		push Y	
PUSH	1001	0110	96	1		push M1	
PUSH	1001	0111	97	1		push M2	
POP	1001	1000	98	1		pop A	
POP	1001	1001	99	1		pop B	
POP	1001	1010	9a	1		pop C	
POP	1001	1011	9b	1		pop D	
POP	1001	1100	9c	1		pop X	
POP	1001	1101	9d	1		pop Y	
POP	1001	1110	9e	1		pop M1	
POP	1001	1111	9f	1		pop M2	

Chapter 2: Numerical Opcode List

	Opcode			Size	Cond. Codes				Assembler		Notes
					C	S	Z	V	Syntax		
MOVI	1010	0000	a0	2					A = expr8		
MOVI	1010	0001	a1	2					B = expr8		
MOVI	1010	0010	a2	2					C = expr8		
MOVI	1010	0011	a3	2					D = expr8		
MOVI	1010	0100	a4	2					X = expr8		
MOVI	1010	0101	a5	2					Y = expr8		
MOVI	1010	0110	a6	2					M1 = expr8		
MOVI	1010	0111	a7	2					M2 = expr8		
ADD16	1010	1000	a8	3					XY = XY ± expr16	2	
ADD16	1010	1001	a9	3					M = XY ± expr16	2	
ADD16	1010	1010	aa	3					XY = M ± expr16	2	
ADD16	1010	1011	ab	3					M = M ± expr16	2	
ADD16	1010	1100	ac	3					XY = expr16	2	
ADD16	1010	1101	ad	3					M = expr16	2	
ADD16	1010	1110	ae	3					XY = A ± expr16	2	
ADD16	1010	1111	af	3					M = A ± expr16	2	
AND	1011	0000	b0	1	0	S	Z	0	A = A & B		
OR	1011	0001	b1	1	0	S	Z	0	A = A B		
XOR	1011	0010	b2	1	0	S	Z	0	A = A ^ B		
NOT	1011	0011	b3	1	0	S	Z	0	A = !A		
ADD	1011	0100	b4	1		C	S	Z	V	A = A + B	
ADDC	1011	0101	b5	1		C	S	Z	V	A = A + B + CY	3
SUB	1011	0110	b6	1		C	S	Z	V	A = A - B	
SUBC	1011	0111	b7	1		C	S	Z	V	A = A - B - CY	4
NEG	1011	1000	b8	1	0	S	Z	V	A = -A		
CLR	1011	1001	b9	1	0	0	1	0	A = 0		
SHL	1011	1010	ba	1		C	S	Z	0	A = A << 1	
SHLC	1011	1011	bb	1		C	S	Z	0	A = A <<< 1	
SHR	1011	1100	bc	1		C	0	Z	0	A = A >> 1	
SHRC	1011	1101	bd	1		C	S	Z	0	A = A >>> 1	
INCR	1011	1110	be	1	0	S	Z	0	A = A + 1		
DECR	1011	1111	bf	1	0	S	Z	0	A = A - 1		

Chapter 2: Numerical Opcode List

	Opcode		Size	Cond. Codes				Assembler		Notes
				C	S	Z	V	Syntax		
LDSP	1100	0000	c0	3					SP = expr16	
LDCODE	1100	0001	c1	1					A = code [XY]	
GOTOXY	1100	0010	c2	1					goto XY	
CALL	1100	0011	c3	3					call expr16	
RET	1100	0100	c4	1					ret	
IN	1100	0101	c5	2					A = in expr8	
OUT	1100	0110	c6	2					out expr8 = A	
LDM	1100	0111	c7	3					A = [M ± expr16]	5
STM	1100	1000	c8	1					[M ± expr16] = A	5
CMP	1100	1001	c9	3	C	S	Z	V	cmp A:B	
ILLEGAL	1100	1010	ca	1						6
ILLEGAL	1100	1011	cb	1						6
ILLEGAL	1100	1100	cc	1						6
ILLEGAL	1100	1101	cd	1						6
ILLEGAL	1100	1110	ce	1						6
ILLEGAL	1100	1111	cf	1						6
BRANCH	1101	0000	d0	3					if == goto expr16	
BRANCH	1101	0001	d1	3					if != goto expr16	
BRANCH	1101	0010	d2	3					if < goto expr16	
BRANCH	1101	0011	d3	3					if <= goto expr16	
BRANCH	1101	0100	d4	3					if > goto expr16	
BRANCH	1101	0101	d5	3					if >= goto expr16	
BRANCH	1101	0110	d6	3					if <u goto expr16	
BRANCH	1101	0111	d7	3					if <=u goto expr16	
BRANCH	1101	1000	d8	3					if >u goto expr16	
BRANCH	1101	1001	d9	3					if >=u goto expr16	
BRANCH	1101	1010	da	3					if !SN goto expr16	
BRANCH	1101	1011	db	3					if SN goto expr16	
BRANCH	1101	1100	dc	3					if !OV goto expr16	
BRANCH	1101	1101	dd	3					if OV goto expr16	
BRANCH	1101	1110	de	3					goto expr16	
BRANCH	1101	1111	df	3					(undefined)	
ILLEGAL	111-	----	e0...ff	1						6

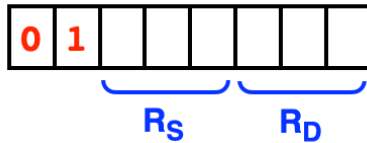
Notes:

1. An instruction “MOV R = R”, where the source and destination registers are the same, is unnecessary and meaningless. Since there is already a perfectly good NOP instruction, such an instruction should never be used. The implementor is free to re-define this opcode. A reasonable implementation is that such an instruction acts exactly as the NOP instruction, i.e., doing exactly what it says.
2. The **imm16** value is added to the register. For this instruction, the assembler syntax allows “...=Reg+expr16”, “...=Reg−expr16”, and “...=Reg”. In the second case, the assembler negates the value, so it is as if the programmer wrote “...=Reg+(−expr16)”. In the third case, it is as if the programmer wrote “...=Reg+0”
3. The CY flag is used to hold the “carry” for multi-byte additions.
4. The CY flag is used to hold the “borrow” for multi-byte subtractions.
5. The **imm16** value is added to M. For this instruction, the assembler syntax allows “[M+expr16]”, “[M−expr16]”, and “[M]”. In the second case, the assembler negates the value, so it is as if the programmer wrote “[M+(−expr16)]”. In the third case, it is as if the programmer wrote “[M+0]”.
6. This opcode is undefined and should be treated as an ILLEGAL instruction. Specific implementations can redefine this opcode as needed.

3: The Instructions

3.1: MOV - Register to Register Move

Instruction Encoding (binary)



where R_S specifies the source register and R_D specifies the destination register.

Opcode (hex)

0x40 ... 0x7f

Instruction Size

1 byte

Description

One byte is copied from source to destination.

Condition Codes

Not modified.

Assembly Code

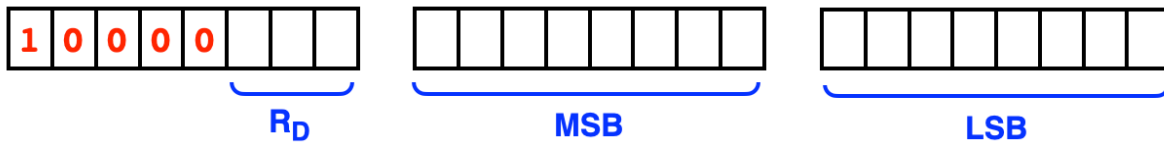
RegD = *RegS*

Example

B = M1 // copy 8 bits from M1 into B

3.2: LD - Load From RAM

Instruction Encoding (binary)



where R_D specifies the destination register. **MSB** is the most significant byte of the address and **LSB** is the least significant byte.

Opcode (hex)

0x80 ... 0x87

Instruction Size

3 bytes

Description

The immediate 16 bit value specifies an address in RAM. One byte is copied from RAM memory to the destination register.

Condition Codes

Not modified.

Assembly Code

Reg = [*expr16*]

Example

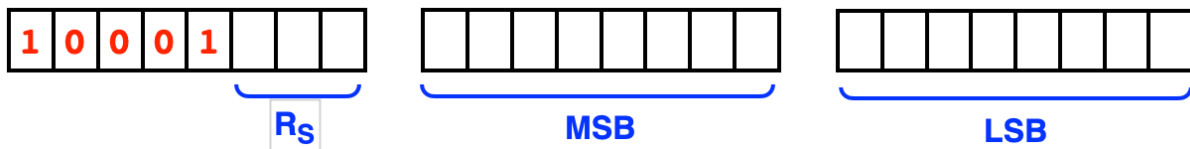
```

C = [MyVar]
...
.ram
MyVar:  .byte 0x42

```

3.3: ST - Store to RAM

Instruction Encoding (binary)



where **Rs** specifies the source register. **MSB** is the most significant byte of the address and **LSB** is the least significant byte.

Opcode (hex)

0x88 ... 0x8f

Instruction Size

3 bytes

Description

The immediate 16 bit value specifies an address in RAM. One byte is copied from the register to RAM memory.

Condition Codes

Not modified.

Assembly Code

`[expr16] = Reg`

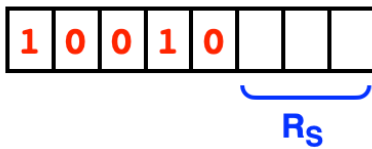
Example

```

[MyVar] = X
...
.ram
MyVar:  .byte 0x0
```

3.4: PUSH - Push Register Onto Stack

Instruction Encoding (binary)



where R_S specifies the source register.

Opcode (hex)

0x90 ... 0x97

Instruction Size

1 byte

Description

The SP register is decremented by 1. Then one byte is copied from the source register to the address in RAM specified by **SP**.

The stack grows downward, from high memory (i.e., larger addresses) to low memory (i.e., smaller addresses). The Stack Pointer (**SP**) register always points to the element at the “top” of the stack.

Condition Codes

Not modified.

Assembly Code

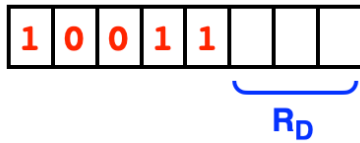
push *Reg*

Example

push A

3.5: POP - Pop Stack Top into Register

Instruction Encoding (binary)



where R_D specifies the destination register.

Opcode (hex)

0x98 ... 0x9f

Instruction Size

1 byte

Description

One byte is retrieved from the RAM from the address held in the **SP** register and moved into the destination register. Then **SP** is incremented by 1.

Condition Codes

Not modified.

Assembly Code

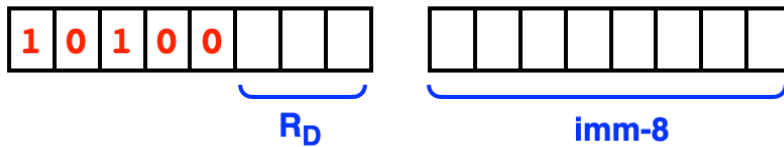
pop *Reg*

Example

pop X

3.6: MOVI - Move Immediate Into Register

Instruction Encoding (binary)



where R_D specifies the destination register and $imm-8$ specifies an immediate 8 bit value.

Opcode (hex)

0xa0 ... 0xa7

Instruction Size

2 bytes

Description

The immediate 8 bit value is copied into the register.

The immediate value is given by an expression. This expression can use decimal and hex constants, symbols defined elsewhere, as well as the usual operators (+, -, *, ...). The final value of the expression must be within -128 ... +255, i.e., 0x00...0xff. The symbols can be defined with the **.equ** assembler directive (as in the example below) or labels, which have as their value a memory address.

Condition Codes

Not modified.

Assembly Code

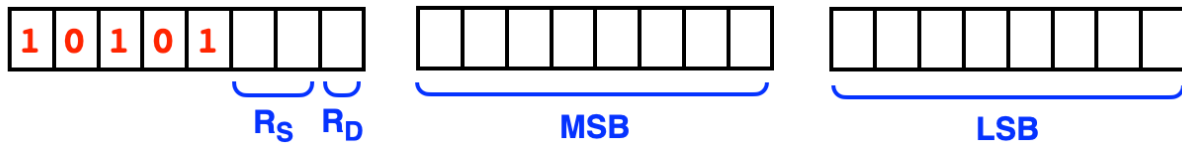
Reg = Expr8

Example

```
A = 0x123
B = MyVal + 34          // Sets B to decimal 234
...
MyVal: .equ 2000/ten    // MyVal == 200
ten:   .equ 10
```

3.7: ADD16 - Add 16 Bit Values

Instruction Encoding (binary)



where R_S specifies the source as follows:

- 00** XY
- 01** M
- 10** <zero>
- 11** A

and R_D specifies the destination register as follows:

- 0** XY
- 1** M

The second and third bytes provide a 16 bit immediate value.

Opcode (hex)

0xa8 ... 0xaf

Instruction Size

3 bytes

Description

If the source is **M**, **XY**, or **A**, the 16 bit immediate value is first added to the value in the register.⁵ If the source field is 10 (i.e., <zero>), there is no addition and the 16 bit value is used directly. In any case, the result is copied into the destination register (either **XY** or **M**). Any overflow with the addition is ignored.

Condition Codes

Not modified.

⁵ If the source is **A**, the 8 bit value in **A** is first zero-extended to 16 bits.

Often **XY** or **M** is used to address memory, for example when stepping through a number of bytes looking for a particular value. This instruction can be used to initialize **XY** or **M** prior to such an operation. Then, this instruction can be used to increment the register as needed on each iteration. For example:

```

    XP = MyString
    call Print
    ...
    // Function to print the null-terminated string pointed to by XY.
Print:
    A = code[XY]
    B = 0
    cmp A:B
    if == goto exit
    XY = XY + 1
    out 0 = A      // Send A to UART
    goto Print
exit:
    ret

```

As a second example, imagine that FLASH memory contains an array of elements and we wish to index into this array. The following code might be used:

```

    A = ...index...      // calculate index i
    XY = A + MyArray     // compute addr of MyArray[i]
    A = code[XY]         // fetch MyArray[i]

```

As a final example, assume **XY** points to a “struct” in memory; this instruction could be used to get at a specific field in the struct, as in:

```

    XY = MyComplexVar
    M = XY + offset40
    A = [M]

```

Assembly Code

Reg = Expr16

Reg = XY

Reg = XY + Expr16

Reg = XY - Expr16

Reg = M

Reg = M + Expr16

Reg = M - Expr16

Reg = A

Reg = A + Expr16

Reg = A - Expr16

Examples

```
M = MyLabel
XY = 0x1234
M = M + 1
M = A + MyLabel
XY = M
```

3.8: AND - Logical And

Instruction Encoding (binary)

1	0	1	1	0	0	0	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xb0

Instruction Size

1 byte

Description

The 8 bits in A are ANDed with the 8 bits in B. The result is stored in A.

Condition Codes

SN, ZE — Modified to reflect the result.

CY, OV — Cleared to zero.

Assembly Code

A = A & B

Example

```
A = [myVar]
B = 0x04          // Check bit 2
A = A & B
if !ZE goto BitWasSet
```

For this and other ALU instructions, only registers A and B can be used. Since there is no variability about which registers are involved, the assembly instruction could — in theory — be reduced to a single token, such as “and”.

Traditionally, assembly code often looked more like this:

```
ld      A,myVar      ; Not EveCore!
movi    B,0x04
and
b.ne    BitWasSet
```

These two examples show the same instruction sequence. EveCore uses an assembly code syntax intended to make the code more human-readable, which is hopefully demonstrated here.

3.9: OR - Logical OR

Instruction Encoding (binary)

1	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xb1

Instruction Size

1 byte

Description

The 8 bits in A are ORed with the 8 bits in B. The result is stored in A.

Condition Codes

SN, ZE — Modified to reflect the result.

CY, OV — Cleared to zero.

Assembly Code

A = A | B

Example

```
A = [myVar]
B = 0x80      // Set bit 7
A = A | B
[myVar] = A
```

3.10: XOR - Logical Exclusive Or

Instruction Encoding (binary)

1	0	1	1	0	0	1	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xb2

Instruction Size

1 byte

Description

The 8 bits in A are XORed with the 8 bits in B. The result is stored in A.

Condition Codes

SN, ZE — Modified to reflect the result.

CY, OV — Cleared to zero.

Assembly Code

A = A ^ B

Example

```
A = [myVar]
B = 0xf0      // Flip upper 4 bits
A = A ^ B
[myVar] = A
```

3.11: NOT - Logical Negation

Instruction Encoding (binary)

1	0	1	1	0	0	1	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xb3

Instruction Size

1 byte

Description

The 8 bits in A are logically complemented. The result is stored in A.

Condition Codes

SN, ZE — Modified to reflect the result.

CY, OV — Cleared to zero.

Assembly Code

```
A = !A
```

Example

```
A = [myVar]
A = !A      // Flip all bits
[myVar] = A
```

3.12: ADD - Addition

Instruction Encoding (binary)

1	0	1	1	0	1	0	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xb4

Instruction Size

1 byte

Description

The value in A is added to the value in B and the result is stored in A.

Condition Codes

CY — Set to 1 iff a carry out of the most significant bit occurred; 0 otherwise.

SN — Set to 1 iff the most significant bit of the result is 1; 0 otherwise.

ZE — Set to 1 iff the result is 0x00; 0 otherwise.

OV — Set to 1 iff signed overflow occurred; 0 if the (signed) result is ok.

Assembly Code

A = A + B

Example

A = [myVar1]

B = [myVar2]

A = A + B

[myResult] = A

The operation of addition is identical, regardless of whether the values are interpreted as signed or unsigned; this is a wonderful property of representing signed values using two's complement. (Other signed representations are mostly historical and no longer in use.)

The only difference arises in the detection of overflow.

If all values are unsigned, then the carry (CY) condition flag will be set if and only if the result is greater than 255 (0xff). For example, adding these two values results in a number larger than 255; it can't be represented in 8 bits.

	1110	1100	(236 decimal)
+	0101	1011	(91 decimal)
1	0100	0111	(327 decimal)

To detect overflow in the case of signed numbers, we must look at the signs of the two operands and of the result. There are exactly two cases in which overflow occurs:

POS + POS ← **Overflow if the result is negative**
NEG + NEG ← **Overflow if the result is positive**

The overflow (OV) condition flag will be set to 1 in exactly these cases, i.e., if and only if the resulting value is mathematically incorrect. If desired, the programmer can insert an instruction to check for overflow.

3.13: ADDC - Addition with Carry

Instruction Encoding (binary)

1	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xb5

Instruction Size

1 byte

Description

The value in A is added to the value in B. If the carry (CY) condition code was set beforehand, 0x01 is also added to the result. The result is stored in A.

Condition Codes

CY, SN, ZE, OV — Modified to reflect the result.

Assembly Code

A = A + B + CY

Example⁶⁷

```

// Compute Result = MyVar1+MyVar2
A = [myVar1+1] // . Add lower byte
B = [myVar2+1] // .
A = A + B      // .
[myResult+1] = A // .
A = [myVar1]   // . Add upper byte
B = [myVar2]   // .
A = A + B + CY // .
[myResult] = A // .
if OV goto Error // Check for signed overflow
```

The ADDC instruction can be used to implement addition of multi-byte numbers, as illustrated by the above example code. First, the ADD instruction is used to add the

⁶ This example assumes that 16 bit values are stored in “big endian” order.

⁷ This example demonstrates a commenting convention whereby dots “.” are used to show which line a particular comment applies to, when a complex operation involves several steps.

least significant bytes. Then, ADDC is used for remaining bytes. After the most significant byte is added, the CY, SN, and OV condition codes will be set to indicate the result of the entire multi-byte operation.

Note that the ZE flag will reflect only whether the most significant byte of the result is zero. To test for zero, all bytes must be individually tested after each ADD/ADDC operation.

3.14: SUB - Subtract

Instruction Encoding (binary)

1	0	1	1	0	1	1	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xb6

Instruction Size

1 byte

Description

The value in B is subtracted from the value in A and the result is stored in A.

Condition Codes

CY — Set to 1 iff a borrow out of the most significant bit occurred;
0 otherwise.

SN — Set to 1 iff the most significant bit of the result is 1; 0 otherwise.

ZE — Set to 1 iff the result is 0x00; 0 otherwise.

OV — Set to 1 iff signed overflow occurred; 0 if the (signed) result is ok.

Assembly Code

A = A - B

Example

```
A = [myVar]      // myResult = myVar - 25
B = 25           //
A = A - B        //
[myResult] = A   //
```

Like addition, the operation of subtraction is identical, regardless of whether the values are interpreted as signed or unsigned. The only difference arises in the detection of overflow.

If the values are unsigned, then we are subtracting one positive number from another. There is only a problem when B is larger than A, in which case the correct answer is a negative number, which cannot be represented as an unsigned number. This occurs exactly when there is a borrow-out from the most significant bit.

Unsigned overflow occurs exactly when the CY flag is set, so CY should be tested to check for overflow.

To detect overflow in the case of signed subtraction, we must look at the signs of the two operands and of the result. There are exactly two cases in which overflow can occur:

$\text{NEG} - \text{POS} = \text{NEG} + \text{NEG} \leftarrow \text{Overflow if the result is positive}$

$\text{POS} - \text{NEG} = \text{POS} + \text{POS} \leftarrow \text{Overflow if the result is negative}$

The overflow (OV) flag will be set to 1 in exactly these cases.

3.15: SUBC - Subtract with Carry (Borrow)

Instruction Encoding (binary)

1	0	1	1	0	1	1	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xb7

Instruction Size

1 byte

Description

The value in A is added to the value in B. If the carry (CY) condition code was set beforehand, 0x01 is also added to the result. The result is stored in A.

Condition Codes

CY, SN, ZE, OV — Modified to reflect the result.

Assembly Code

A = A - B - CY

Example

```

                                // Result = MyVar1-MyVar2
A = [myVar1+1]                 // . Subtract lower byte
B = [myVar2+1]                 // .
A = A - B                      // .
[myResult+1] = A               // .
A = [myVar1]                   // . Subtract upper byte
B = [myVar2]                   // .
A = A - B - CY                 // .
[myResult] = A                 // .
if CY goto Error               // Check for unsigned overflow

```

The SUBC instruction can be used to implement subtraction of multi-byte numbers, in a way similar to ADDC. First, the SUB instruction is used to subtract the least significant bytes. Then, SUBC is used for remaining bytes. After the most significant byte is subtracted, the CY, SN, and OV condition codes will be set to indicate the result of the entire multi-byte operation.

3.16: NEG - Arithmetic Negation

Instruction Encoding (binary)

1	0	1	1	1	0	0	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xb8

Instruction Size

1 byte

Description

The value in A is treated as a signed value; it is negated.

Condition Codes

CY — Cleared to 0.

SN — Modified to reflect the result.

ZE — Modified to reflect the result.

OV — Set to 1 if A was 0x80; cleared to 0 otherwise.

Assembly Code

A = -A

Example

```
A = [myVar]           // Negate myVar
A = -A                //
[myResult] = A        //
```

With two's complement representation, a value can be negated by first flipping the bits and then adding 1. Negating 1 gives 0xFF, which is the representation of -1:

$0x01 \rightarrow 0xFE; 0xFE + 1 \rightarrow 0xFF$

Negating 0 gives 0:

$0x00 \rightarrow 0xFF; 0xFF + 1 \rightarrow 0x00$

Overflow occurs in exactly one situation, namely negating the most negative number, 0x80 (i.e., -128 decimal). The largest positive number is +127, so there is a problem:

$0x80 \rightarrow 0x7F; 0x7F + 1 \rightarrow 0x80$

In this case, A will be unchanged and the OV condition code will be set to 1.

There is no single instruction to test a value to see if it is zero or negative. All ALU instructions set the ZE and SN flags, so there is not much need. However if such a

test is needed, a sequence of two NEG instructions in a row will do the trick, with the added benefit of testing against 0x80 with the OV flag:

```
// Test A without changing it
A = -A
A = -A
if SN goto isNegative
if ZE goto isZero
// must be positive
```

You might consider the NEG;NEG sequence to be a single “TEST” instruction. That hypothetical instruction is 2 bytes long, which is not too long for an instruction that is rarely needed.

3.17: CLR - Clear Register

Instruction Encoding (binary)

1	0	1	1	1	0	0	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xb9

Instruction Size

1 byte

Description

Register A is set to 0x00.

Condition Codes

CY — Cleared to 0.

SN — Cleared to 0.

ZE — Set to 1.

OV — Cleared to 0.

Assembly Code

A = 0

Example

```
A = 0           // myVar = 0
[myVar] = A
```

You can use this instruction if you wish to clear the CY, SN, or ZE condition code flags. In order to set the ZE flag, use this sequence:

```
A = 0
A = A + 1
```

3.18: SHL - Shift Left

Instruction Encoding (binary)

1	0	1	1	1	0	1	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xba

Instruction Size

1 byte

Description

The bits in A are shifted left by one bit. The most significant bit is moved into CY. A 0 bit is shifted into the least significant bit position.

Condition Codes

CY — Set to the previous value of A₇.

SN, ZE — Modified to reflect the result.

OV — Cleared to 0.

Assembly Code

```
A = A << 1
```

Example

```
A = [myVar]      // myVar = myVar * 4
A = A << 1
A = A << 1
[myVar] = A
```

Note that a left shift is equivalent to multiplying by 2, as long as the sign bit doesn't change. This works for negative numbers, too.

There is no multi-bit shift; only one bit at a time.

3.19: SHLC - Shift Left With Carry

Instruction Encoding (binary)

1	0	1	1	1	0	1	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xbb

Instruction Size

1 byte

Description

The bits in A are shifted left by one bit. The most significant bit is moved into CY. The previous value of CY is shifted into the least significant bit position. This instruction can be used to implement multi-byte shifting, similarly to ADDC and SUBC.

Condition Codes

CY — Set to the previous value of A₇.
 SN, ZE — Modified to reflect the result.
 OV — Cleared to 0.

Assembly Code

A = A <<< 1

Example

```
A = [myVar+1]    // Shift a 16 bit value left one bit
A = A << 1        // . Shift LSB; CY hold bit to be transferred
[myVar+1] = A     // .
A [myVar+0]      // . Shift MSB, using CY from LSB
A = A <<< 1       // .
[myVar+0] = A     // .
```

The assembler uses this notation:

```
<<    Shift Left
<<<   Shift Left (with CY in)
>>    Shift Right
>>>   Shift Right (with CY in)
```

When the carry is shifted in, there is “more”; think of the extra < as meaning “more”.

EveCore contains no ROTATE instruction. However, the operation can be achieved with a few instructions. For example, to rotate A right by one bit, use the following instructions:

```
push A
A = A << 1      // Sets CARRY to bit 7
pop A
A = A <<< 1     // Shifts CARRY (i.e. bit 7) in from right
```

Alternatively, you could save A in a register, if you concerned about execution speed.

A similar code sequence can be used to rotate to the right.

3.20: SHR - Shift Right

Instruction Encoding (binary)

1	0	1	1	1	1	0	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xbc

Instruction Size

1 byte

Description

The bits in A are shifted right by one bit. The least significant bit is moved into CY. A 0 bit is shifted into the most significant bit position.

Condition Codes

CY — Set to the previous value of A₀.

ZE — Modified to reflect the result.

SN, OV — Cleared to 0.

Assembly Code

```
A = A >> 1
```

Example

```
A = [myVar]      // myVar = myVar / 4
A = A >> 1
A = A >> 1
[myVar] = A
```

Note that a left shift is equivalent to dividing by 2, as long as the number is either positive or unsigned. This does not work for negative numbers, since a 0 is shifted into the sign bit.

3.21: SHRC - Shift Right With Carry

Instruction Encoding (binary)

1	0	1	1	1	1	0	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xbd

Instruction Size

1 byte

Description

The bits in A are shifted right by one bit. The least significant bit is moved into CY. The previous value of CY is shifted into the most significant bit position. This instruction can be used to implement multi-byte shifting, similarly to ADDC and SUBC.

Condition Codes

CY — Set to the previous value of A₀.
 SN, ZE — Modified to reflect the result.
 OV — Cleared to 0.

Assembly Code

A = A >>> 1

Example

```
A = [myVar+0]    // Shift a 16 bit value right one bit
A = A >> 1       // . Shift MSB; CY hold bit to be transferred
[myVar+0] = A     // .
A [myVar+1]      // . Shift LSB, using CY from MSB
A = A >>> 1      // .
[myVar+1] = A     // .
```

When shifting a multi-byte value left, you must start with the least significant byte and work toward the more significant (lefthand) bytes. However, when shifting right, you must start with the most significant byte and work to the right.

3.22: INCR - Increment

Instruction Encoding (binary)

1	0	1	1	1	1	1	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xbe

Instruction Size

1 byte

Description

Register A is incremented.

Condition Codes

CY — Cleared to 0.

SN, ZE — Modified to reflect the result.

OV — Cleared to 0.

Assembly Code

A = A + 1

Example

```
A = C           // Increment register C
A = A + 1       //
C = A           //
```

To increase A by an amount other than 1, use a sequence such as this:

```
B = 5
A = A + B
```

3.23: DECR - Decrement

Instruction Encoding (binary)

1	0	1	1	1	1	1	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xbf

Instruction Size

1 byte

Description

Register A is decremented.

Condition Codes

CY — Cleared to 0.

SN, ZE — Modified to reflect the result.

OV — Cleared to 0.

Assembly Code

A = A - 1

Example

```

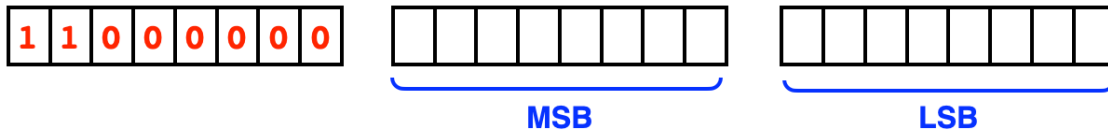
D = 47                // LOOP 47 times
Loop:                // .
...                  // Do something
A = D                // ENDLOOP
A = A - 1            // .
D = A                // .
if !ZE goto Loop    // .

```

This is a classic example of looping. Note that the DECR instruction will modify the ZE flag, while the MOV instruction right after it does not modify the condition codes. Thus, the BRANCH instruction will work properly.

3.24: LDSP - Load Stack Pointer

Instruction Encoding (binary)



where the second and third bytes specify a 16 bit value, with **MSB** being the most significant byte.

Opcode (hex)

0xc0

Instruction Size

3 bytes

Description

The **SP** register is loaded with the immediate 16 bit value.

Condition Codes

Not modified.

Assembly Code

`SP = expr16`

Example

```

        SP = MyStkTop    // This stack capacity is 256 bytes
        ...
        .ram
MyStk:   .skip 0x100
MyStkTop:

```

The Stack Pointer (**SP**) register is nominally 16 bits. The size of RAM is assumed to be a power of 2, such as 4 KB. If RAM is smaller than 64 KB, the higher bits are unneeded, unused, and need not be implemented. Any extra bits in the immediate value are ignored when loading **SP**.

It is customary to place the stack at the very top of RAM. The proper initialization is 0x0000; the initial PUSH operation will first decrement **SP** to 0xFFFF and then write the pushed value to the uppermost byte in RAM. Initializing **SP** to 0x0000 will also work if the size of the RAM is less than 64 KB.

If a particular implementation guarantees that the **SP** register is always initialized to 0x0000 at power-on-reset, then executing the LDSP instruction might be unnecessary. However, the programmer should never assume such initialization and should invoke LDSP at the beginning of any program that will use the PUSH, POP, CALL, or RET instructions.

There is no way to retrieve the current value of **SP**; why would you want to, anyway? The only other instructions that use **SP** are PUSH, POP, CALL, and RET.

3.25: LDCODE - Load Byte From Flash Memory

Instruction Encoding (binary)

1	1	0	0	0	0	0	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xc1

Instruction Size

1 byte

Description

Read a byte from the memory and copy it into the A register. XY contains the address to read from. If the program/code/flash memory is distinct from the data memory, the data is fetched from the code memory.

Condition Codes

Not modified.

Assembly Code

```
A = code[XY]
```

Example

```
A = ...index...
XY = A + MyTable    // Get the i-th element from an array
A = code[XY]
.flash
MyTable:
.byte 34
.byte -7
.byte 92
...
```

Von Neumann Architecture Implementation

The EveCore may be implemented with a single, unified memory holding both programs and data. In such an implementation, this instruction provides a way to load bytes indirectly using the XY register. This augments the LDM instruction, which uses the M register for the address.

For example, here is code to copy a block of bytes:

```
loop:  A = code[XY]
       [M] = A
       XY = XY + 1
       M = M + 1
       ...
       if ... goto loop
```

Harvard Architecture Implementation

If the EveCore is implemented with separate memories for program and data, the LDCODE instruction accesses program/code/flash memory, while other instructions (LD, LDM, ST, STM, ...) access the data memory.

The EveCore has no instruction to retrieve data from flash in which the address to use is included in the instruction as a simple immediate value. Such an instruction might look like this:

```
A = code[0x1000]    // Not a real instruction
```

The reason is simple. Presumably the programmer knows what is in flash memory — after all, flash is fixed and known at programming time — so any needed value can be loaded directly using a MOVI instruction. For example:

```
A = myConstant
...
myConstant: .equ    123
```

Note that the hypothetical LOAD instruction from above would require 3 bytes, while the MOVI instruction is just 2 bytes long, i.e., a direct move is more concise.

However, the flash might contain an array or table of some sort, as the above example suggests. For this reason, the LDCODE instruction, as it actually exists, is necessary.

3.26: GOTOXY - Jump Indirect Through Register

Instruction Encoding (binary)

1	1	0	0	0	0	1	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xc2

Instruction Size

1 byte

Description

The value in register XY is copied into the Program Counter (PC).

Condition Codes

Not modified.

Assembly Code

goto XY

Example

```

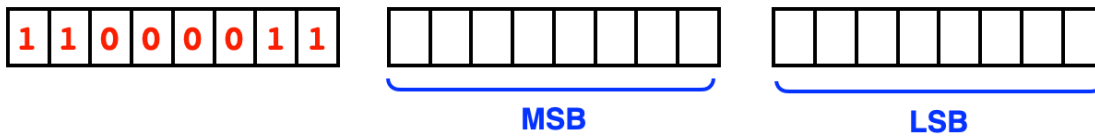
    A = ...index...           // Compute index i
    A = A << 1                 // Multiply by 4
    A = A << 1                 //
    XY = A + JumpTable        // and jump through the table
    goto XY                    //
JumpTable:
    goto foo                   // A GOTO is 3 bytes. Make each
    .skip 1                    // table entry 4 bytes
    goto bar
    .skip 1
    goto myCode
    .skip 1
    ...

```

Indirect “jumps” can be used for a variety of programming techniques, such as implementing “switch” statements and dynamic dispatching in object oriented programs.

3.27: CALL - Invoke Function

Instruction Encoding (binary)



where the second and third bytes specify a 16 bit target address, with **MSB** being the most significant byte.

Opcode (hex)

0xc3

Instruction Size

3 bytes

Description

Invoke the function by pushing the return address onto the stack and jumping to the location specified by the 16 bit immediate address.

More precisely, this instruction does the following:

SP $\leftarrow$ **SP**-2

RAM [SP] $\leftarrow$ **MSB(RetAddr)**

RAM [SP+1] $\leftarrow$ **LSB(RetAddr)**

PC $\leftarrow$ **imm16**

where **RetAddr** is the address of the instruction following the CALL instruction.

Condition Codes

Not modified.

Assembly Code

`call expr16`

Example

```
...  
call foo  
...
```

```
foo:
```

```
...  
ret
```

How are arguments to be passed to functions? This document does not promote a particular “calling convention”. But we can say it is always good practice to clearly document the following in a block comment at the top of each function:

- What arguments are passed, and in which registers.
- What value, if any, is returned, and in which register(s).
- Which registers will be modified (i.e., “trashed”) by the function.
- Which registers will be untouched (or restored if modified) by the function.
- How are registers used within the function.

In microcontroller coding, in order to optimize register usage and minimize code size, a programmer will often pass arguments in different, virtually inscrutable patterns. This makes thoroughly documenting register usage absolutely critical.

If a register must be used within a function, but the calling code expects that register to be unmodified by the function, the programmer can use the PUSH instruction to save the register’s contents at the top of the function and the POP instruction to restore register before return. For example:

```
foo:  
    push M1            // Save regs  
    push M2            //  
    ...code using M...  
    pop M2             // Restore regs... in reverse order!  
    pop M1             //  
    ret
```

Often, functions use **stack frames** to hold the function’s local variables. The compiler will generate code to allocate and free these frames as functions are called and return. This approach supports recursive functions quite naturally.

EveCore is a microcontroller architecture and instructions to facilitate access to a stack frame were not included. (This makes recursive programs more difficult to implement, but ask yourself: does a typical embedded control application really need recursion?)

Note that functions that are not recursive have no need for stack frames and can store any “local” variables at fixed addresses in memory.

In the following example, the function MyFunc uses two variables. To emphasize that these variables are used only within MyFunc, the programmer has placed them directly at the top of the code for MyFunc and given them suggestive names. The assembler allows you to switch back and forth between flash and ram, so this works just fine.

```
MyFunc:
    .ram
mf_var1: .byte 0
mf_var2: .byte 0
    .flash
    ...body of MyFunc, which uses mf_var1 and mf_var2...
    ret

... other functions...
```

3.28: RET - Return From Function

Instruction Encoding (binary)

1	1	0	0	0	1	0	0
---	---	---	---	---	---	---	---

Opcode (hex)

0xc4

Instruction Size

1 byte

Description

Return from a function by popping the return address off the stack and jumping back to it.

More precisely, this instruction does the following:

$\text{MSB}(\text{PC}) \leftarrow \text{RAM}[\text{SP}]$

$\text{LSB}(\text{PC}) \leftarrow \text{RAM}[\text{SP}+1]$

$\text{SP} \leftarrow \text{SP}+2$

Condition Codes

Not modified.

Assembly Code

ret

Example

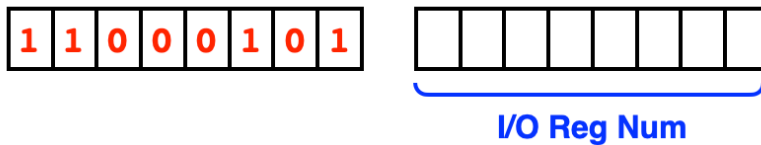
foo:

...

ret

3.29: IN - Read From I/O Register

Instruction Encoding (binary)



where **I/O Reg Num** is an immediate 8 bit value specifying the I/O register.

Opcode (hex)

0xc5

Instruction Size

2 bytes

Description

EveCore accommodates up to 256 I/O registers, which can be read or written individually in order to control and interact with peripheral devices such as digital I/O pins, UARTs, switches, LEDs, etc. This instruction transfers one byte from the I/O register specified by the immediate 8 bit value to A.

Condition Codes

Not modified.

Assembly Code

A = in *expr8*

Example

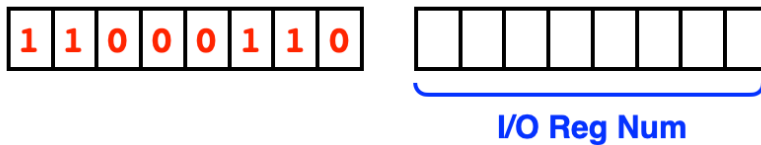
A = in 0x01

The numbers and definitions of the I/O registers will vary from implementation to implementation. For this reason, as well as a form of documentation and to increase portability of programs, it is customary to use symbolic names for I/O registers. The following code is equivalent to the above example, but obviously clearer:

```
A = in UART_recv_reg
...
UART_recv_reg: .equ 0x01
```

3.30: OUT - Write To I/O Register

Instruction Encoding (binary)



where **I/O Reg Num** is an immediate 8 bit value specifying the I/O register.

Opcode (hex)

0xc6

Instruction Size

2 bytes

Description

This instruction transfers one byte from A to the I/O register specified by the immediate 8 bit value.

Condition Codes

Not modified.

Assembly Code

out *expr8* = A

Example

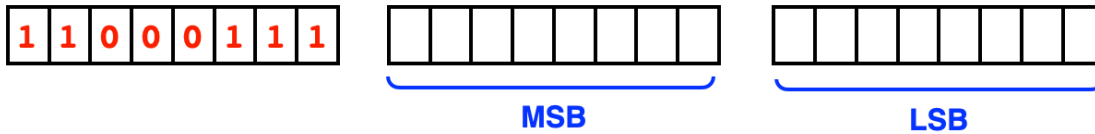
```
out UART_xmit_reg = A
...
UART_xmit_reg: .equ 0x01
```

Note that the same register number can be used for both input and output, perhaps serving quite different functions. In the examples here and for the IN instruction, address 0x01 is used for both sending and receiving. Writing (OUT) to the register transmits the character in A, while reading (IN) from the register fetches a recently received character.⁸ As another example, a single register number might be used for both control and status: reading would return the device status, while writing would send a control command to the device.

⁸ This is only an example. Consult device-specific documentation to determine what register 0x01 is actually used for.

3.31: LDM - Read From RAM Using M

Instruction Encoding (binary)



where the second and third bytes form an immediate 16 bit value (*imm16*) with **MSB** being the most significant byte.

Opcode (hex)

0xc7

Instruction Size

3 bytes

Description

A byte is read from RAM and copied into register **A**. The address is formed by adding the value of **M** to the immediate 16 bit value.

Condition Codes

Not modified.

Assembly Code

```
A = [M]
A = [M + expr16]
A = [M - expr16]
```

Example

```
M = ...startAddr...
MyLoop:
  A = [M]
  M = M + 1
  ...
  if ... goto MyLoop
```

If the address is in **M** and no offset from **M** is needed, then the assembler accepts the following form and generates machine code in which **imm16** is zero:

```
A = [M]
```

It might be the case that **M** contains the address of a block of data and the programmer wishes to load from a specific offset within that block. Here is an example:

```
M = MyData_B
A = [M+4]
...
.ram
MyData_A: .skip 10
MyData_B: .skip 10
MyData_C: .skip 10
```

On the other hand, the roles may be reversed. In other words, **M** will contain an index or offset into a block of data, instead of the address of the block itself. If the block of data is located at a fixed address in RAM, you can use code like this:

```
M = ...index...
A = [M+MyArray]
...
.ram
MyArray: .skip 100
```

The programmer can use the subtraction form whenever a negative offset is needed:

```
A = [M - expr16]
```

Here is an example:

```
A = [M - 3*x]
```

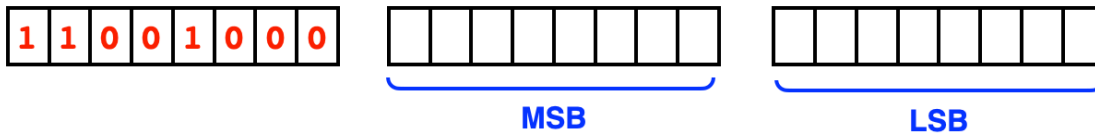
This is assembled identically to the following:

```
A = [M + -(3*x)]
```

In other words, the **imm16** value will be automatically negated at assembly-time so the addition at runtime will yield the expected result.

3.32: STM - Store Into RAM Using M

Instruction Encoding (binary)



where the second and third bytes form an immediate 16 bit value (*imm16*) with **MSB** being the most significant byte.

Opcode (hex)

0xc8

Instruction Size

3 bytes

Description

The byte in **A** is written to RAM. The address is formed by adding the value of **M** to the immediate 16 bit value.

Condition Codes

Not modified.

Assembly Code

```
[M] = A
[M + expr16] = A
[M - expr16] = A
```

Example

```
M = ...startAddr...
MyLoop:
  A = ...something...
  [M] = A
  M = M + 1
  ...
  if ... goto MyLoop
```

The comments under LDM also apply to STM.

3.33: CMP - Compare A to B

Instruction Encoding (binary)

1	1	0	0	1	0	0	1
---	---	---	---	---	---	---	---

Opcode (hex)

0xc9

Instruction Size

1 byte

Description

The values in **A** and **B** are compared and the condition codes are set accordingly. More precisely, the condition codes are set in exactly the same way as they would be for

$$A = A - B$$

except that the result of the subtraction is discarded and **A** is unchanged.

Condition Codes

CY — Set to 1 iff $A < B$ using unsigned comparison; 0 otherwise.

SN — Set to 1 iff $A-B$ is negative using signed subtraction; 0 otherwise.

ZE — Set to 1 iff $A=B$; 0 otherwise.

OV — Set to 1 iff $A-B$ overflows using signed subtraction; 0 otherwise.

Assembly Code

```
cmp A : B
```

Examples

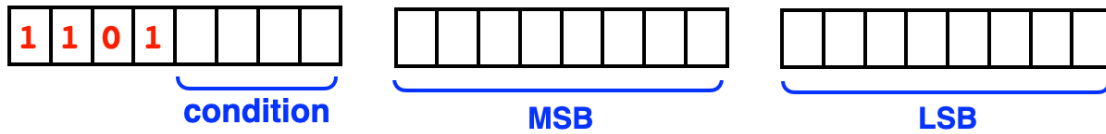
```
A = [M]
B = '\n'
cmp A:B
if == goto FoundNL

A = ...
B = 100
if <u goto NinetyNine_or_less
```

See the BRANCH instruction for a discussion of various conditions.

3.34: BRANCH - Conditional GOTO

Instruction Encoding (binary)



where **condition** specifies the condition to be tested. The second and third bytes specify a 16 bit **target** address, with **MSB** being the most significant byte of the address.

Opcode (hex)

0xd0 ... 0xdf

Instruction Size

3 bytes

Description

The condition specified by **condition** field is tested. If the condition is true, control is transferred to the **target** location, otherwise execution continues with the next instruction.

Condition Codes

The condition codes are not modified.

Assembly Code

```
if condition goto target
goto target
```

Examples

```
cmp A:B
if <=u goto myLabel

A = A - 1
if ZE goto itsZero

goto Loop
```

Condition Field Encoding

Here is the encoding of the *condition* field:

	<u>Condition</u>	<u>Assembler Notation</u>	
0000	ZE	== <u>or</u> ZE	
0001	!ZE	!= <u>or</u> !ZE	
0010	SN^OV	<	<i>signed <</i>
0011	(SN^OV) ZE	<=	<i>signed ≤</i>
0100	(SN=OV) & !ZE	>	<i>signed ></i>
0101	SN=OV	>=	<i>signed ≥</i>
0110	CY	<u <u>or</u> CY	<i>unsigned <</i>
0111	CY ZE	<=u	<i>unsigned ≤</i>
1000	! (CY ZE)	>u	<i>unsigned ></i>
1001	!CY	>=u <u>or</u> !CY	<i>unsigned ≥</i>
1010	!SN	!SN	
1011	SN	SN	
1100	!OV	!OV	
1101	OV	OV	
1110	1	< <i>branch always taken</i> >	
1111	< <i>not defined</i> >		

The assembler recognizes the following character sequences as a single token:⁹

```
<u
<=u
>u
>u
```

The programmer can test for equality in either of these ways:

```
if ZE goto myLabel
if == goto myLabel
```

Although both instructions perform the same test and both assemble to the same machine code, one might be clearer than the other, depending on circumstances, as in:

```
A = A >> 1
if ZE goto itIsNowZero
...
CMP A:B
if == goto theyAreEqual
```

⁹ No space is allowed within tokens like "<=u". (This is a bit unusual. In other languages, something like "<=u" would be parsed as two tokens: "<=" followed by the identifier "u".) On the other hand, conditions such as "!ZE" comprise two tokens, so writing"! ZE" is fine.

Likewise the programmer can choose between **!=** and **!ZE**, between **<u** and **CY**, and between **>=u** and **!CY**.

You can understand the conditions for unsigned comparison this way:

If **B** is larger than **A**, then **A-B** would be a negative number. With subtraction, this means that the **CY** (i.e., borrow) will be set to 1. So whenever **A** and **B** are interpreted as unsigned values, **CY** is true if and only if **A < B**. Thus, the condition for **<u** is simply **CY=1**.

Naturally, we can “or” in the **ZE** flag to obtain the condition for **<=u** ($\leq$ unsigned). Then, noting that **<** is the opposite of $\geq$, we can negate the condition for **<u** to obtain the condition for **>=u**, i.e., **!CY**. Likewise, the condition for **>u** is simply the negation of the condition for **<=u**.

Understanding the conditions to test signed **<**, **$\leq$** , **>**, and **$\geq$** is more complex.

First, consider the sign bits of **A** and **B**. Let’s make a table and include the **SN** and **OV** results for all possible combinations of positive and negative values of **A** and **B** and the result **A-B**.

$A - B = R$				SN	OV	<	SN $\oplus$ OV
$A < B$	NEG -7	NEG -3	NEG	1	0	1	1
	NEG -7	NEG -9	POS	0	0	0	0
$A < B$	NEG	POS	NEG	1	0	1	1
$A < B$	NEG	POS	POS	0	1	1	1
	POS	NEG	NEG	1	1	0	0
	POS	NEG	POS	0	0	0	0
$A < B$	POS 7	POS 9	NEG	1	0	1	1
	POS 7	POS 3	POS	0	0	0	0

Note that we have overflow when a negative minus a positive results in a positive value, and when a positive minus a negative gives yields a negative value. These are reflected in the OV column.

In the “<” column, we have written the result we desire.

If one operand is negative and the other is positive the desired **<** result is obvious. However, when the signs of **A** and **B** are the same, then we have two cases, one where **<** holds and one where it does not hold. We’ve shown this in the chart above using specific example numbers, like **-7** and **9**. Fortunately, these cases are exactly distinguished by the sign of the result!

In summary, given the **SN** and **OV** condition flags resulting from the subtraction, we can see that the desired $<$ result is simply **SN xor OV**.¹⁰

The $\leq$ relationship follows by ORing in the **ZE** flag. The $>$ and $\geq$ relationships follow from negating $\leq$ and $>$, respectively.¹¹

One *condition* (namely 1110) is **always true**. If this is used in the instruction, the jump will always occur.¹² Thus, the simple, unconditional “goto” is a special case of the conditional BRANCH instruction. In this case, the programmer writes:

```
goto myLabel
and not
if true goto myLabel
```

The CMP instruction is only useful for comparing 8 bit values. To compare values that are larger than 8 bits, the programmer must use the SUB and SUBC instructions. For example, to compare 16 bit values, the following code will first perform the subtraction, ignoring the result. Then, the conditional BRANCH instructions can be used as needed.

For example, assume that two halfword values are stored in **myValA** and **myValB**:

```
A = [myValA+1]      // Subtract the LSB and set BORROW
B = [myValB+1]      //
A = A - B            //
A = [myValA+0]      // Subtract the MSB, using BORROW
B = [myValB+0]      //
A = A - B - CY       //
if < goto aLessThanb // Jump if myValA < myValB (signed)
```

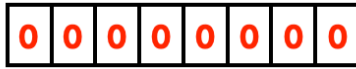
Some architectures specify branch targets using **PC-relative addressing**. In such a system, the address is given as an offset which is added to the address of the instruction itself, to give the address to jump to. This can save bits when the offset is limited. For example, in some architectures the offset might be limited to 8 bits while the entire memory uses 48 bit addresses. To keep things simple, the EveCore does not use PC-relative addressing.

¹⁰ By “positive” we really mean “non-negative”, but “positive” seems clearer and doesn’t affect our reasoning. (When **A** and **B** are equal, the subtraction result is zero — i.e., a “positive” number — and the $<$ relationship does not hold. Since **SN** and **OV** are cleared to 0 whenever the operands are equal, **SN xor OV** results in 0, as desired.)

¹¹ Recall that the opposite of **XOR** is the **EQUAL** Boolean function. Also, don’t forget DeMorgan’s Laws, including $\sim(X | Y) = \sim X \& \sim Y$.

¹² An affinity for symmetry might suggest the final, **undefined** condition should be **false**, in which case the jump would never be taken. But three **NOPs** in a row would get the job done and waste just as much memory. Perhaps some implementors will find a reasonable use for this encoding.

3.35: NOP Instruction

Instruction Encoding (binary)**Opcode (hex)**

0x00

Instruction Size

1 byte

Description

This instruction does nothing.

Condition Codes

Not modified.

Assembly Code

nop

Example

nop

This instruction might be useful in the circuit design process or in introducing specific timing delays.

4: Assembly Code

4.1: Introduction

This chapter documents the canonical **EveCore Assembly Language**. This is the language accepted by an assembler I created.¹³

As with other assemblers, each machine instruction and each assembler directive must appear entirely on a single line and each line may contain at most one instruction or directive.

In languages like C and C++, the end-of-line is just another white space character. However here (like in Python), the end-of-line is a unique lexical token, relevant in parsing.

However, there are some unusual features of EveCore Assembler. For example, there are two styles of comments:

```
// ...through end-of-line
/* ... */
```

Traditionally, assemblers have used a variety of commenting convention, such as:

```
# ...through end-of-line
; ...through end-of-line
```

Today, most programmers are familiar with the commenting introduced by “C”.

Comments nest.¹⁴

¹³ The assembler is written in “C” and the source code is free to all to use. The same source file compiles and runs on both Mac OS X and Windows. The program consists of a single stand-alone file which is called **asm8.c**. It only relies on #include files that are commonly available. It can be compiled with, e.g., “**gcc -std=c99 -Wall -O2 asm8.c -o asm8**”

¹⁴ Unlike the “C” language, the following is processed as a single comment:

```
/* ... /* ... */ ... */
```

Also note that // style comments naturally nest, as they do in “C”:

```
//
// // Initialize A
// A = 0x7b /* decimal 123 */
//
```

4.2: Source and Output Files

The assembler accepts a single source file, which typically has a “**.s**” extension. This is specified on the command line. There is no “**#include**” directive.¹⁵

The command line option “**-h**” prints info about various other options.

The command line option “**-h2**” provides a brief summary of the syntax accepted by the assembler.

The assembler will produce a single output file, but there are several possible output formats in which this file can be written. The format of the output file is selected by using one of these command line options:

- l
- hex1
- hex2
- sv

¹⁵ The assembler does not perform any “macro preprocessing”. However, programmers are free to preprocess files in any way they want before they get to the assembler.

Listing Output

The first output format is “listing format”, which produces a file in human-readable form, making it easy to see how the bytes are placed in memory. This format is selected with “-l” (el). For example, this command line:

```
% asm8 echo.s -l -o echo.txt
```

will produce a file such as the following:

```

0000          1: // Example program
              2:          .flash
              3:
0000          4: Main:
0000 c0 00 00    5:          SP = 0x0000          // Stack begins at highest RAM addr
0003 ac 00 16    6:          XY = msg              // Print greeting
0006 c3 00 33    7:          call Print            //
0009          8: Loop:
0009 c5 00        9:          A = in UART_rcv        // Echo 1 byte on UART
000b c6 00       10:         out UART_xmit = A      //
000d a1 71       11:         B = 'q'                // if "q" then goto Main
000f c9          12:         cmp A:B                //
0010 d0 00 00    13:         if == goto Main        //
0013 de 00 09    14:         goto Loop             // else continue echoing
              15:
0016 0a456368    16: msg:      .string "\nEcho Program\nPlease type...\0"
001a 6f205072
001e 6f677261
0022 6d0a506c
0026 65617365
002a 20747970
002e 652e2e2e
0032 00

              17:
              18:
              19: // Print Function
              20: //      passed: XY = pointer to NUL-terminated string in FLASH
              21: //      uses:   A, B, XY
0033          22: Print:
0033 c1          23:         A = code[XY]
0034 a8 00 01    24:         XY = XY + 1
0037 a1 00       25:         B = 0
0039 c9          26:         cmp A:B
003a d0 00 42    27:         if == goto PrintExit
003d c6 00       28:         out UART_xmit = A
003f de 00 33    29:         goto Print
0042          30: PrintExit:
0042 c4          31:         ret
              32:
              33: UART_xmit: .equ 0          // I/O register for sending
              34: UART_rcv:  .equ 0          // I/O register for receiving

```

FLASH Segment Size: 67 (0x43) bytes

RAM Segment Size: 0 (0x0) byte

Basic Hex Output

The second output format can be selected with the “**-hex1**” command line option. This is the default, so these two command lines do the same thing.

```
% asm8 echo.s -hex1 -o echo.txt
% asm8 echo.s      -o echo.txt
```

In either case, the file will contain only the bytes. It will look like this:

```
c0
00
00
ac
00
16
c3
00
33
c5
00
c6
00
a1
71
...
```

Hex Output With Comments

The third output format is specified with the “**-hex2**” command line option. Here is a sample of that format:

```

// 1:      // Example program
// 2:      // .flash
// 3:
// 4:      Main:
c00000 // 5:      SP = 0x0000      // Stack begins at highest RAM addr
ac0016 // 6:      XY = msg        // Print greeting
c30033 // 7:      call Print      //
// 8:      Loop:
c500   // 9:      A = in UART_recv // Echo 1 byte on UART
c600   // 10:     out UART_xmit = A //
a171   // 11:     B = 'q'         // if "q" then goto Main
...
```

The “**-delim**” command line option can be used to change the commenting character(s) from the default “//”. For example:

```
% asm8 echo.s -hex2 -delim # -o echo.txt
```

System Verilog Output

The final output format is specified with the “-sv” command line option, which produces a sequence of Verilog assignments. Here is a sample of that format:

```

//      1:  // Example program
//      2:  .flash
//      3:
//      4:  Main:
16'h0000: data = 8'hc0; //      5:      SP = 0x0000          // Stack ... highest RAM addr
16'h0001: data = 8'h00;
16'h0002: data = 8'h00;
16'h0003: data = 8'hac; //      6:      XY = msg          // Print greeting
16'h0004: data = 8'h00;
16'h0005: data = 8'h16;
16'h0006: data = 8'hc3; //      7:      call Print          //
16'h0007: data = 8'h00;
16'h0008: data = 8'h33;
//      8:  Loop:
16'h0009: data = 8'hc5; //      9:      A = in UART_recv      // Echo 1 byte on UART
16'h000a: data = 8'h00;
16'h000b: data = 8'hc6; //     10:      out UART_xmit = A      //
16'h000c: data = 8'h00;
16'h000d: data = 8'ha1; //     11:      B = 'q'              // if "q" then goto Main
16'h000e: data = 8'h71;
...

```

The idea is that the output of the assembler can be copy-pasted directly into Verilog code such as the following, in order to create a purely combinational read-only memory module:

```

module ROM_MEMORY (
    input  logic [15:0] addr,
    output logic [7:0]  data_out
);
    logic [7:0] data;
    assign data_out = data;
    always_comb
        case (addr [15:0])

```

<<< Paste Here >>>

```

//      1:  // Example program
//      2:  .flash
//      3:
//      4:  Main:
16'h0000: data = 8'hc0; //      5:      SP = 0x0000          // Stack ...
16'h0001: data = 8'h00;
16'h0002: data = 8'h00;
<<< ...etc... >>>

```

```

        default: data = 8'hff;
    endcase
endmodule // ROM_MEMORY

```

The “**-var**” command line option can be used to change the variable name from the default “data”. For example:

```
% asm8 echo.s -sv -var myDataName -o echo.txt
```

4.3: Assembler Directives

Here are the assembler directives:

.flash	Subsequent bytes will be placed in FLASH memory
.ram	Subsequent bytes will be placed in RAM memory
.byte	Defines an 8 bit data item to be placed in memory
.halfword	Defines a 16 bit data item to be placed in memory
.word	Defines a 32 bit data item to be placed in memory
.string	Defines a sequence of bytes/characters to be placed in memory
.skip	Causes N bytes to be skipped over and left undefined
.equ	Equates a symbol with the value of an expression

A **label** is an identifier and is equated (by the assembler) to a specific 16 bit address. When defined, a label is followed by a colon. The label can appear alone on a line or may precede an instruction or directive on the same line. The label is defined to be the address of the instruction that immediately follows it.¹⁶ The colon is not part of the label and does not appear when the label is used as an operand.

¹⁶ Except for **.equ** as discussed later.

4.4: The **.flash** and **.ram** Directives

If the EveCore implementation uses a “Von Neumann” architecture — i.e., if there is only a single, unified memory — then you should just place a **.flash** directive at the beginning of the source file. You should not use the **.ram** directive.¹⁷

On the other hand, in the EveCore implementation uses a “Harvard” architecture — i.e., if the program and data memories are separate — the **.flash** and **.ram** directives can be used to control where subsequent source code should be placed. The **.flash** directive instructs the assembler to place subsequent material in the FLASH memory. Similarly, the **.ram** directive instructs the assembler to place subsequent material in the RAM memory.

A **.flash** directive must appear before any instruction. A **.flash** or **.ram** directive must appear before any data definition.

There can be multiple occurrences of **.flash** and **.ram** directives in a single source file; the programmer does not have to group all the instructions together in one place and group all the data definitions together in another place. The bytes intended for FLASH and RAM can thus be intermixed in the source file. In other words, the use of **.flash** and **.ram** directives partition the source file into FLASH and RAM sections and everything must be in one of these.

Only the following may appear in a RAM section:

.byte	Defines an 8 bit data item in memory
.halfword	Defines a 16 bit data item in memory
.word	Defines a 32 bit data item in memory
.skip	Causes N bytes to be skipped over and left undefined
.equ	Equates a symbol with an expression

There should be no initializing value for **.byte**, **.halfword**, or **.word** directives, since RAM is not initialized on power-on-reset.¹⁸ However, **.byte**, **.halfword**, and **.word** directives must have initializing expressions whenever they are used within a FLASH section.

¹⁷ Perhaps the **.flash** directive should be renamed. For a Von Neumann implementation, both program and data will go into a unified memory, which must not be read-only flash if you want to update data variables. Perhaps “program” and “data” would have been better choices.

¹⁸ An initial value of 0 elicits a warning. Any other value triggers an error.

4.5: The .byte Directive

The **.byte** directive allocates a byte in memory.

If a **.byte** directive appears in a FLASH section, it must be followed by a single expression, which gives the 8 bit value to be placed in FLASH.

If the **.byte** directive appears in a RAM section, there should be no initializing expression and (presumably) the byte in RAM will be explicitly initialized by instructions when the program begins execution.

The **.byte** directive may or may not be preceded by a label, either on the same line or not.

The initializing expression may contain decimal, hex numbers, and ASCII character constants. The usual operators may be used. The computations are performed using 32 bits and the finally value is checked to make sure it will fit in 8 bits. The allowable range is 0xFFFF_FF80 ... 0x0000_000FF. In decimal, this is -128 ... +255.

Here are examples:

```
valA:  .byte -100           // 0x96, signed 156
valB:  .byte (symX+10) >> 2
       .byte 0xe8           // unsigned 232, signed -24
myNL:  .byte '\n'
symX:  .equ  0x4c
```

Note that the definition of a symbol need not appear earlier in the source file than its uses. In the above example, **symX** is used on the second line but defined on the fifth line

4.6: The **.halfword** and **.word** Directives

The **.halfword** and **.word** directives are identical to the **.byte** directive, except in the amount of memory they allocate:

.byte	8 bits
.halfword	16 bits
.word	32 bits

The value of the expression must fall into the legal ranges, which are given elsewhere in this document.

4.7: The **.skip** Directive

The **.skip** directive requires an operand which specifies the number of bytes to be allocated in memory. A block of this many bytes will be placed in memory.

Generally speaking, there will be a label directly preceding the **.skip** directive, and this label will be set to the address of the first byte of the allocated chunk of bytes. For example:

```
MyArray: .skip 30
```

Below is another example in which a block of 100 bytes is set aside for a stack. The value of **myStackTop** will be **myStack**+100, i.e., the address of the byte directly after the block of memory labeled **myStack**:

```
...  
SP = myStackTop  
...  
.ram  
myStack:  
    .skip 100  
myStackTop:
```

If the **.skip** occurs in the FLASH section, the bytes would be set aside but they would remain forever fixed and unchangeable; after all, it's FLASH memory. It seems pointless to waste a bunch of FLASH memory this way.

Therefore, the **.skip** is normally used only in the RAM section. The **.skip** doesn't imply the block will be initialized to zero on power-on-reset. Whether or not the RAM is initialized is an implementation detail.

4.8: The .string Directive

The **.string** directive takes as an operand a string of characters enclosed in double quotes. For example:

```
msg1: .string "hello 😊😊😊😊😊 there!\r\n\0"
```

This directive may not occur in the RAM section.

The string may contain any Unicode character and the UTF-8 encoding is used.

Every ASCII character is represented with a single byte. The ASCII character set ranges over 0x00 ... 0x7f. The values 0x20...0x7e are printable. The values 0x00...0x1f, and 0x7f are “control characters” and are not directly printable.

There are several approaches to encoding the Unicode character set, including UTF-8. The ASCII character set is a proper subset of the UTF-8 encoding. In particular, every ASCII string is a legal UTF-8 string and every UTF-8 string containing only ASCII characters is indistinguishable from straight ASCII text.

The usual escape sequences may be used for the common ASCII control characters:

\0	= 0x00	= NUL	= ctrl-@
\a	= 0x07	= ALERT	= ctrl-g
\b	= 0x08	= BACKSPACE	= ctrl-h
\t	= 0x09	= TAB	= ctrl-i
\n	= 0x0a	= NEWLINE	= ctrl-j
\v	= 0x0b	= VERTICAL TAB	= ctrl-k
\f	= 0x0c	= FORMFEED	= ctrl-l
\r	= 0x0d	= RETURN	= ctrl-m
\e	= 0x1b	= ESCAPE	= ctrl-[
\d	= 0x7f	= DELETE.	

Also, \xHH can be used for an arbitrary hex value.

4.9: The `.equ` Directive

Normally, a label is defined as the address of the item that follows it. For example **msg** and **MyLabel** in:

```
msg:      .string  "Hello"
          ...
MyLabel:
          A = 42
          M = msg
          ...
```

However, the `.equ` directive can be used to associate an arbitrary value with a symbol.

Here are examples:

```
four:     .equ  five-1
five:     .equ  5
anAddr:   .equ  MyLabel
addrB:    .equ  (MyLabel+0x100)*five
```

The `.equ` directive must be preceded by a symbol and a colon, which must on the same line.

This code:

```
x:
  four:     .equ  five-1
  msg:      .string  "Hello"
```

would define **x** to be the address of the string, i.e., the address of the 'H' character.

We use the term “label” to refer to a symbol that happens to be an address. For example, **MyLoop** in:

```
MyLoop:
    ...
    goto MyLoop
```

Otherwise, we use the term “symbol”. For example, **arrSize** in:

```
arrSize    .equ    10
myArr:     .skip   arrSize
           ...
           B = arrSize
           ...
```

However, the assembler does not make any distinction between labels and symbols, so the following code is legal, although meaningless and certainly wrong:

```
B = MyLoop    // Move an address into B???
...
goto arrSize  // Jump to location 10???
```

This architecture contains no PC-relative addresses. All code is placed in a single source file and there is no linking of multiple object files. Therefore, there is no distinction between “relative”, “relocatable”, and/or “absolute” values. All values are absolute numbers. Likewise there is no distinction between labels (i.e., relative addresses) and symbols (i.e., absolute values).

5: Discussion

5.1: Motivation

The EveCore architecture is intended to be a straightforward microcontroller with these goals:

- **Implementable in Verilog** on a small FGPA
- Understandable by first year **Computer Science students**
- Serve as a **tutorial on core design**

Existing microcontrollers have lengthy, marginally comprehensible documentation. One goal is a design whose **documentation is simple** and straightforward.

Another goal is to be **usable in real-world microcontroller applications**. Thus, EveCore is more than just a minimal CPU. There are many simple computer designs (e.g., the “Ben Eater” computer), but these are only useful in teaching basic principles and not practical for actual deployment or use.

5.2: Comparison With 6502 / 8051 / AVR

The EveCore architecture is at the level of these well-known microcontrollers. Similarities include:

- Accumulator-based operation
- 8 bit registers
- No support for an OS kernel
- Variable length instructions of 1 or a few bytes

Here are some differences:

- EveCore emphasizes ISA simplicity over performance or die utilization, with the aim of making programming and FPGA design easier and more enjoyable.
- Memories are now cheaper and larger, so complicated encoding schemes intended to minimize memory usage are now less critical.

- Transistor counts are now much higher, so the tradeoff between minimizing circuitry versus orthogonality of the ISA has shifted.
- Execution performance is now dominated by cycles per instruction, rather than memory access latency, especially when flash and RAM are integrated on the same chip as the core.
- Circuit design using Verilog / VHDL facilitates otherwise complex and error-prone circuit design, reducing the demand for minimality.
- EveCore has no interrupt mechanism.
- Some instructions are left as exercises for you.
- There is no mechanism for brown-out detection or a watchdog timer.
- There are no built-in timers/counters.
- Specifications related to peripheral I/O devices are not included in the EveCore ISA.
- The EveCore assembly language improves on traditional assembly notation, while still remaining a true assembly language for expressing machine code.
- EveCore avoids Little Endian ordering.
- There are no instructions to support Binary Coded Decimal (BCD) or half-carries.

Many microcontrollers have complicated addressing schemes where, for example, locations in a particular block of memory (e.g., a “zero page”) can be addressed more efficiently. They usually include truncated addresses with limited range and/or PC-relative addressing. Some include instructions to access individual bits. EveCore opts for uniform 16 bit addressing throughout.

Many microcontrollers combine the ability to LOAD/STORE from/to memory with the ability to increment/decrement a memory pointer. In EveCore, this requires two instructions.

5.3: Functionality You Might Want to Add

You are free to build on this architecture — it’s open and free. Here are ideas for what could be added:

- Multiply Instruction — Maybe signed and unsigned versions.
- Other Instructions — rotate, divide, compare-immediate, ...
- Single byte instructions to LOAD/STORE that also perform M++ at the same time. Maybe also XY++/M--/--M.
- An interrupt mechanism.
- Instructions to support Binary Coded Decimal (BCD).

- Various peripheral devices, such as Pulse Width Modulation(PMW) or Analog to Digital Conversion (ADC).
- Timers/counters.

5.4: What Was Left Out Of EveCore

A number of features are simply not appropriate for this sort of architecture.

- Larger registers / more registers
- General purpose registers (EveCore is strictly accumulator-based)
- Ability to save the condition codes
- Interrupts / Exceptions / Trap Handling
- System Calls and User / Kernel modes
- Privilege instructions and any form of protection
- Virtual Memory / Page Tables
- Synchronization Primitives (e.g., CAS, FENCE instructions)
- Cacheing concerns

EveCore is a microcontroller and will never be capable of running an Operating System kernel. 🙄

EveCore could reasonably support “C” code. 😊

EveCore is at roughly the level of the AVR architecture and — as Arduino programmers know — “C” can be compiled into AVR machine code. Certain operations (e.g., 32 bit signed division) would likely compile to calls to hand-written functions from a support library.

All of the above features are included in x86-64, ARM, and Risc-V, as well as an another architecture that I designed, which is called **Blitz-64**. But that’s another matter.

Document Revision History

Version numbers are not used to identify revisions to this document. Instead the date and the author’s name is used. The history of this document is:

<u>Date</u>	<u>Author</u>
13 March 2024	Harry H. Porter III <document created>
23 May 2024	Harry H. Porter III
2 July 2024	Harry H. Porter III <current version>

In the spirit of the open-source and free software movements, the author grants permission to freely copy and/or modify this document, with the following requirement:

- *On the first (title) page, you must add your name and modify the date.*
- *You must append your date/name to the revision history.*
- *You must not alter this section, except to add to the revision history.*

Any material lifted should be referenced.

Please contact the author if you find...

- Inaccurate information that you can correct
- Incomplete information that you can fill in
- Confusing text that needs to be reworded

Change History Log

- 16 March 2024:** Initial version completed
- 10 April 2024:** Editing and cleanup
- 8 May 2024:** Editing and cleanup
- 23 May 2024:** Revised section 4.2 to match asm8 changes
- 2 July 2024:** Added section “Instruction Encoding Summary”; minor formatting